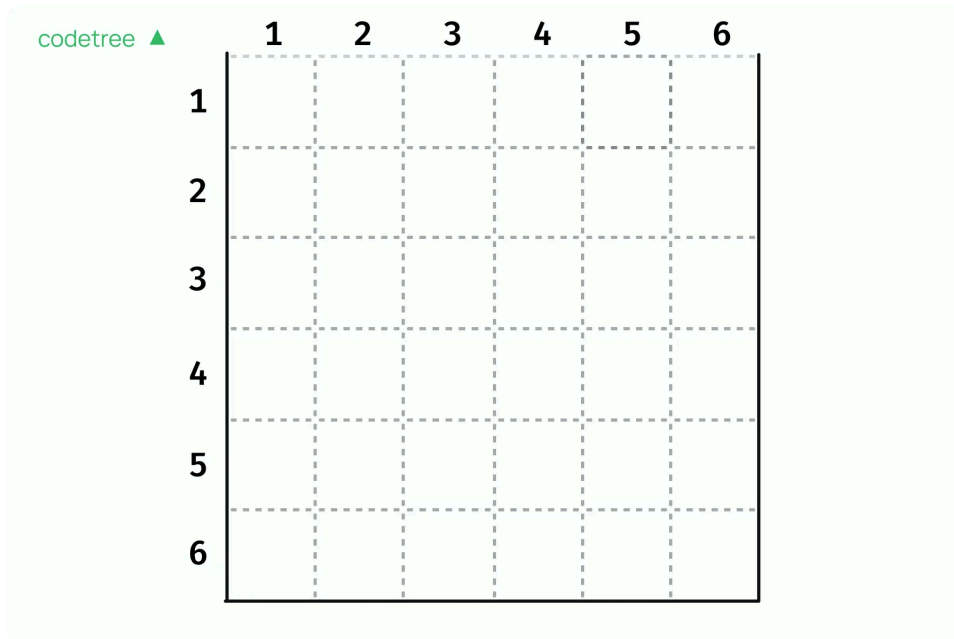


코드트리는 회사 확장을 위해 택배 사업을 시작합니다. 알고리즘 회사의 이점을 이용해서 완벽한 택배 하차 알고리즘을 만들었습니다.

[5][5]

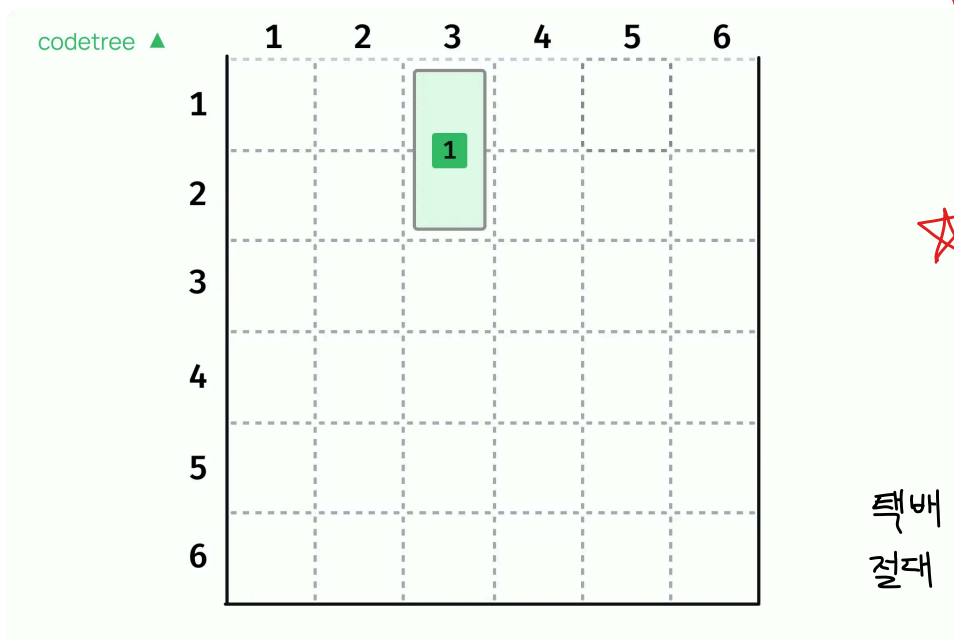
택배는 $N \times N$ 크기의 정사각형 형태의 격자에서 진행됩니다.



택배 하차는 다음과 같은 방법으로 진행됩니다.

1. 택배 투입 1번 동작

- 택배는 직사각형 모양으로, 직사각형 왼쪽 열의 위치 c , 가로 크기(w)와 세로 크기(h)로 주어집니다. 각각의 택배는 택배 번호(k)를 가집니다.



택배 구조체

- $x(c)$
- $y(r)$
- w
- h
- k

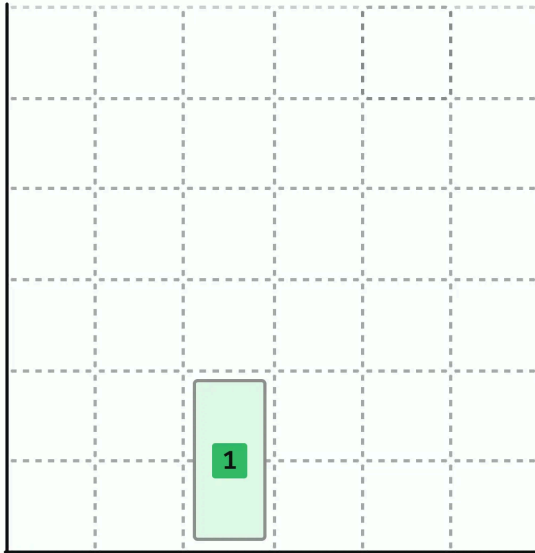
☆ 택배 구조체를 보고
`int board[5][5]`에
택배 모양 차점 번호 저장.
⇒ 지버깅 등에 용이. ☆

택배 구조체만 가지고 계산만 하는 것?
절대 금지. 실생활 문제.
실제 시뮬레이션 필수.

자주 나오는 유형

- 투입된 택배들은 중력에 의해 하단으로 떨어집니다.

1-1번 동작

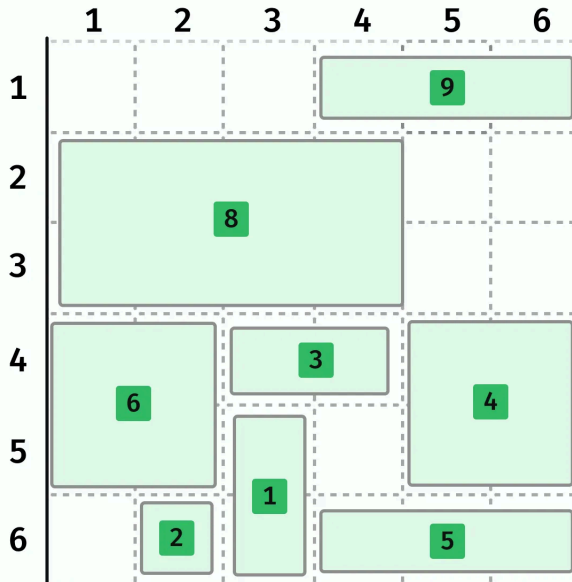


- 그대로 떨어져서 바닥으로 닿거나 내려가던 중 다른 짐을 만나면 거기서 멈춥니다.

1 / 4

- 택배 투입이 완료되었을 때, 어떠한 택배들도 공간을 벗어나지 않음이 보장됩니다.

codetree ▲

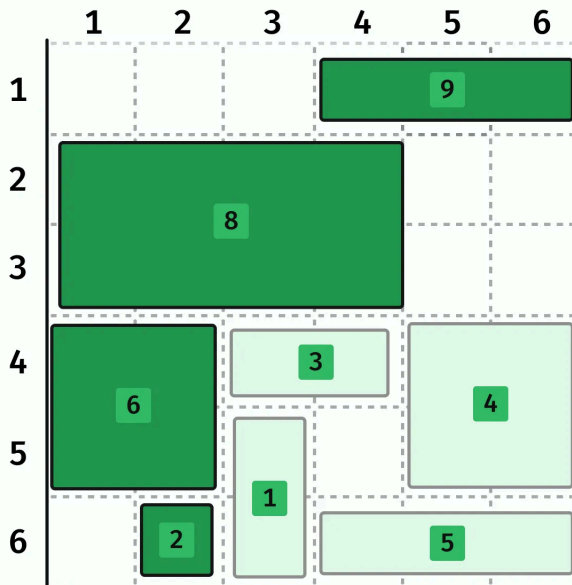


2. 택배 하차 (좌측) 2번 동작

- 쌓인 택배 중에 잡고 왼쪽으로 이동했을 때 다른 택배와 부딪히지 않고 뺄 수 있는 택배를 먼저 하차합니다. 2-1번 동작

조건

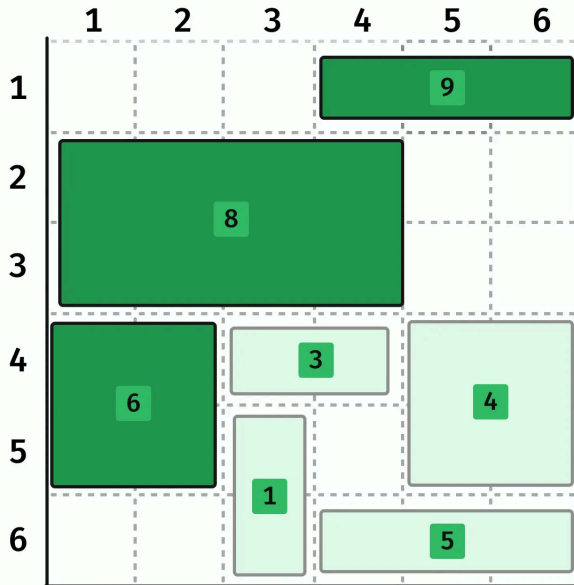
codetree ▲



- 그러한 택배가 여러개일 경우, 택배 번호(k)가 작은 택배를 먼저 하차합니다.

operator <

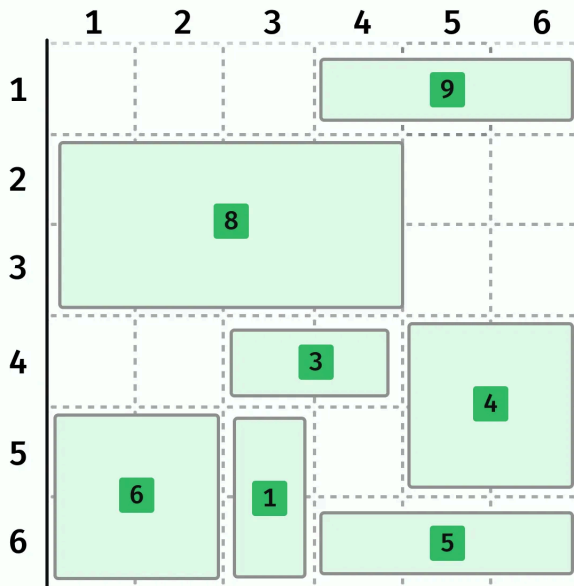
codetree ▲



- 해당 택배를 하차한 이후, 공간에 남아있는 택배들 중 떨어질 수 있는 것들은 떨어집니다.

2-2번 동작

codetree ▲



중력이 작용해서 내려오는 동작이
1번과 2번에서 중복.
굳이 따로 만들 필요 X.

3. 택배 하차 (우측) 3번 동작

- 2의 과정을 우측에서도 진행합니다. 3-1번 동작

while(packageCnt)

공간에 있는 택배를 모두 하차할 때까지 2, 3의 과정을 반복합니다.

다음과 같은 방법으로 택배를 하차할 때, 하차되는 택배의 번호를 순서대로 출력하는 프로그램을 작성하세요.

입력

```
for (1 ≤ k ≤ M) {
    택배 투입(k);
    중력 작용(k);
}
```

제한 조건

- $2 \leq N \leq 50$ dir 넘겨주면
- $1 \leq M \leq 100$ 뺄 수 있는 번호 찾는 함수랑 하차 함수에 대해서 좌측, 우측 분리 필요 X.
- $1 \leq k \leq 100$
- $1 \leq w \leq N$
- $1 \leq h \leq N$ 하차 함수는 옮길 필요 없이 그냥 없애면 됨
- $1 \leq c \leq N$
- 택배 번호 k는 서로 겹치지 않습니다.

while (모두 하차 전까지) {

k = 좌측으로 뺄 수 있는 번호();

좌측 하차();

전체 중력 작용();

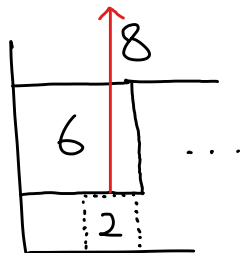
k = 우측으로 뺄 수 있는 번호();

우측 하차();

전체 중력 작용();

→ 전체 중력 작용도 새로운 함수 불필요. (그 열만)

출력



← 방금 빠진 택배의 수직 위로 탐색하면서 걸리는 것들만 해주면 됨. (내 그림 참고)
또는 귀찮으면 그냥 진짜 전체 다 실행
for (1 ≤ k ≤ M) 중력 작용(k);

입력 예제

2가 빠졌으면 2가 차지하는 열만 위로 보면 됨.
6과 8이 걸리므로, 중력 작용(6), 중력 작용(8)

예제 1

입력

```
6 8
1 2 1 3
2 1 1 2
6 2 2 1
5 1 3 4
4 2 2 5
3 1 2 3
```

8 2 4 1
9 1 3 4

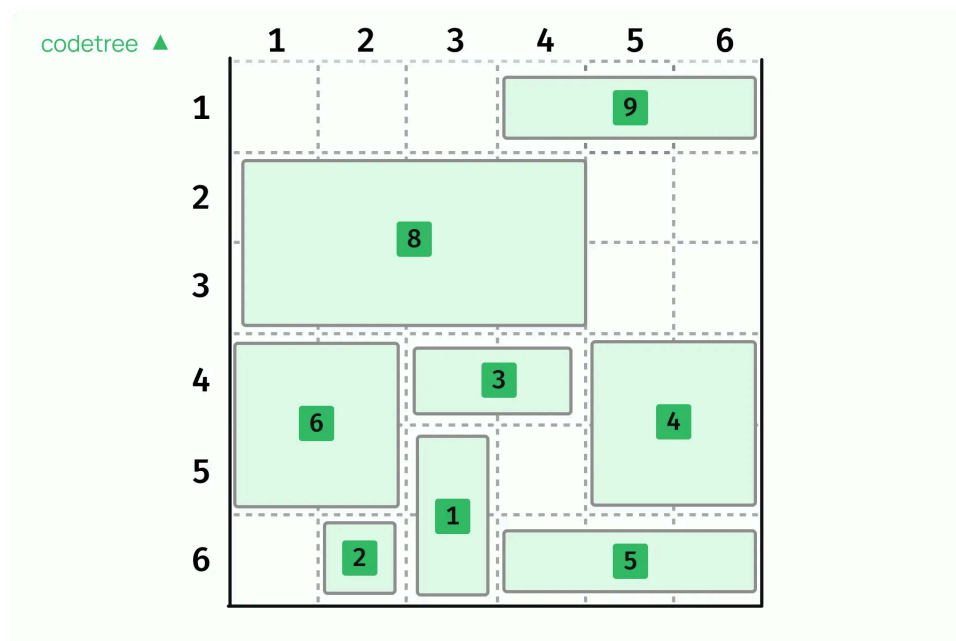
출력

2
4
3
5
6
1
8
9

예제 설명

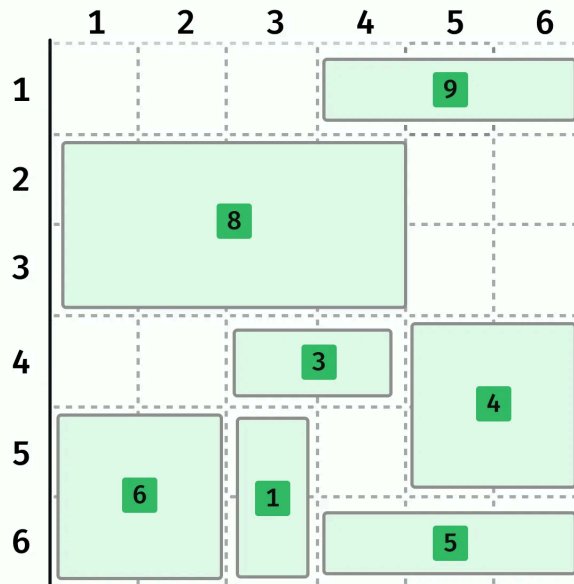
▼ 접기

택배를 모두 쌓은 이후의 모습



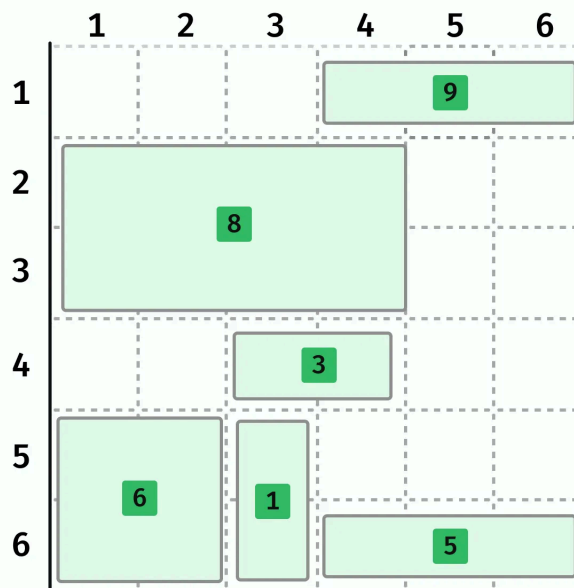
왼쪽 하차 (2번)

codetree ▲



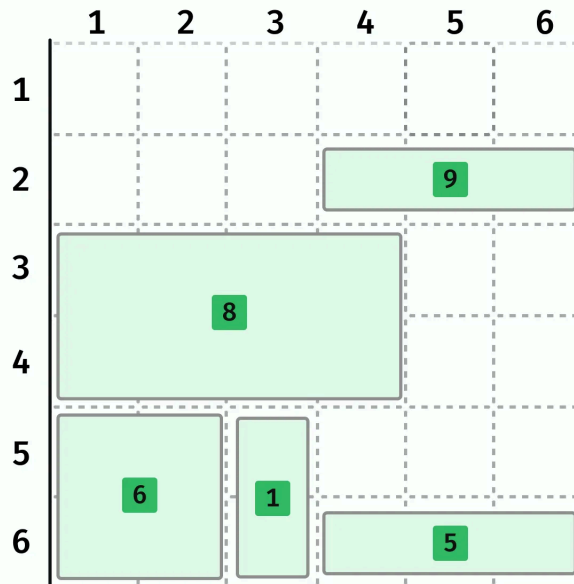
오른쪽 하차 (4번)

codetree ▲



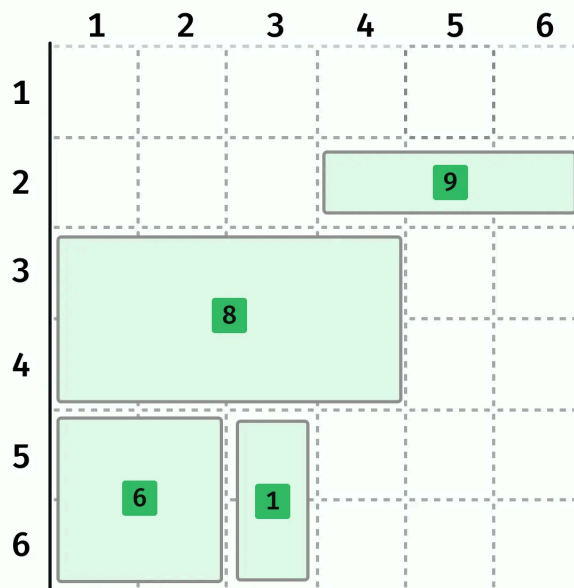
왼쪽 하차 (3번)

codetree ▲



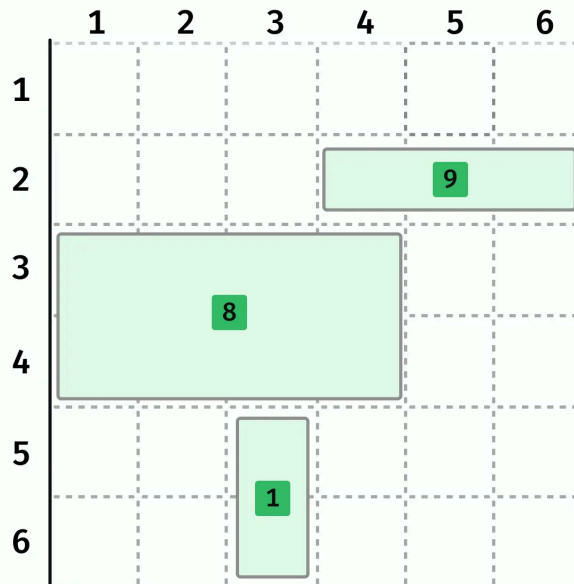
오른쪽 하차 (5번)

codetree ▲



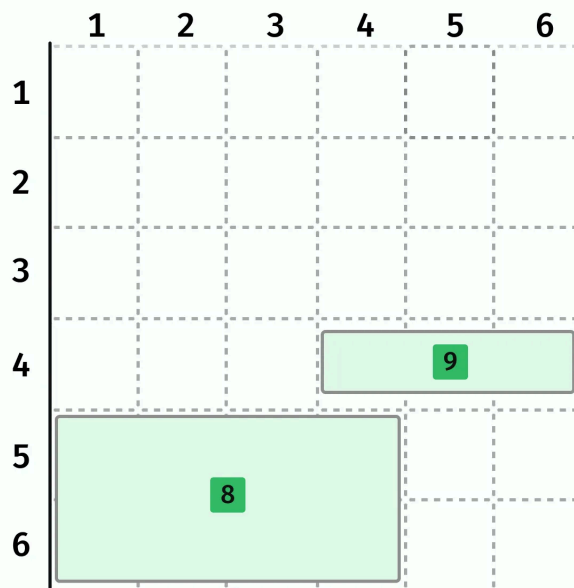
왼쪽 하차 (6번)

codetree ▲



오른쪽 하차 (1번)

codetree ▲



왼쪽 하차 (8번)

codetree ▲

	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

9

오른쪽 하차 (9번)

codetree ▲

	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

예제 2

입력

```
10 10
5 8 2 7
12 9 1 10
13 1 2 4
14 10 1 6
19 1 1 9
3 7 5 1
6 1 1 4
8 2 2 2
```

9 1 1 8
1 1 1 8

출력

3
1
8
12
6
9
13
19
14
5

제한